

torch.baddbmm#

<https://pytorch.org/docs/stable/generated/torch.baddbmm.html>

Description:

Performs a batch matrix-matrix product of matrices in batch1 and batch2. input is added to the final result. batch1 and batch2 must be 3-D tensors each containing the same number of matrices. If batch1 is a $(b \times n \times m)$ tensor, batch2 is a $(b \times m \times p)$ tensor, then input must be broadcastable with a $(b \times n \times p)$ tensor and out will be a $(b \times n \times p)$ tensor. Both alpha and beta mean the same as the scaling factors used in torch.addbmm(). If beta is 0, then the content of input will be ignored, and nan and inf in it will not be propagated. For inputs of type FloatTensor or DoubleTensor, arguments beta and alpha must be real numbers, otherwise they should be integers.

Function Signature

```
torch.baddbmm(input, batch1, batch2, out_dtype=None, *,  
beta=1, alpha=1, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

beta (Number, optional) – multiplier for input (β) alpha (Number, optional) – multiplier for $\text{batch1} @ \text{batch2}$ (α) out (Tensor, optional) – the output tensor.

Code Examples

```
>>> M = torch.randn(10, 3, 5)  
>>> batch1 = torch.randn(10, 3, 4)  
>>> batch2 = torch.randn(10, 4, 5)  
>>> torch.baddbmm(M, batch1, batch2).size()  
torch.Size([10, 3, 5])
```

Detailed Documentation

Documentation

Performs a batch matrix-matrix product of matrices in batch1 and batch2. input is added to the final result.

batch1 and batch2 must be 3-D tensors each containing the same number of matrices.

If batch1 is a $(b \times n \times m)(b \times n \times m)(b \times n \times m)$ tensor, batch2 is a $(b \times m \times p)(b \times m \times p)(b \times m \times p)$ tensor, then input must be broadcastable with a $(b \times n \times p)(b \times n \times p)(b \times n \times p)$ tensor and out will be a $(b \times n \times p)(b \times n \times p)(b \times n \times p)$ tensor. Both alpha and beta mean the same as the scaling factors used in torch.addbmm().

If beta is 0, then the content of input will be ignored, and nan and inf in it will not be propagated.

For inputs of type FloatTensor or DoubleTensor, arguments beta and alpha must be real numbers, otherwise they should be integers.

This operator supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

input (Tensor) – the tensor to be added

batch1 (Tensor) – the first batch of matrices to be multiplied

batch2 (Tensor) – the second batch of matrices to be multiplied

out_dtype (dtype, optional) – the dtype of the output tensor, Supported only on CUDA and for torch.float32 given torch.float16/torch.bfloat16 input dtypes

beta (Number, optional) – multiplier for input (β)

alpha (Number, optional) – multiplier for batch1@batch2 (α)

out (Tensor, optional) – the output tensor.